HAMK
Häme University
of Applied Sciences

Lecture 1
Supervised Learning

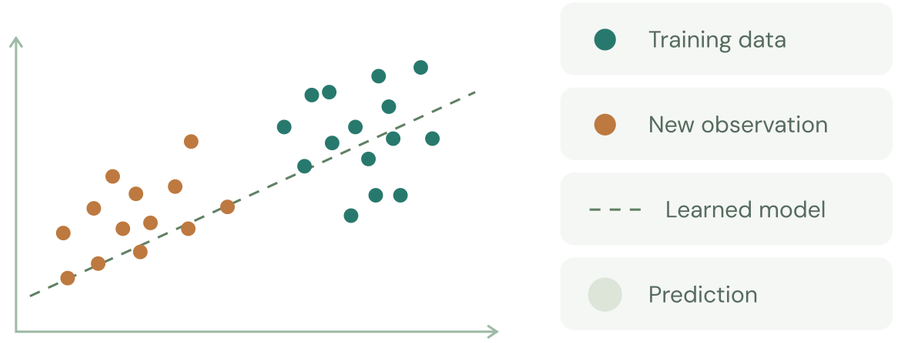
Predictive Analytics with Python

What is Machine Learning, Supervised Learning, Predictive Analytics, k-NN

Mazhar Mohsin

From data to predictions

Learn how supervised learning turns data into useful predictions.



Model performance (example)

Compare how different models perform.



Learning outcomes

By the end of this lecture, you should be able to:

1. Explain machine learning and its relationship to predictive analytics.
2. Distinguish supervised learning from unsupervised and reinforcement learning.
3. Differentiate classification and regression problems.
4. Describe the end-to-end predictive analytics process, including data preparation and evaluation.
5. Explain the intuition, steps, distance metrics, and key hyperparameters of k-nearest neighbours (k-NN).
6. Select appropriate evaluation metrics and recognize common modelling pitfalls such as data leakage and overfitting.

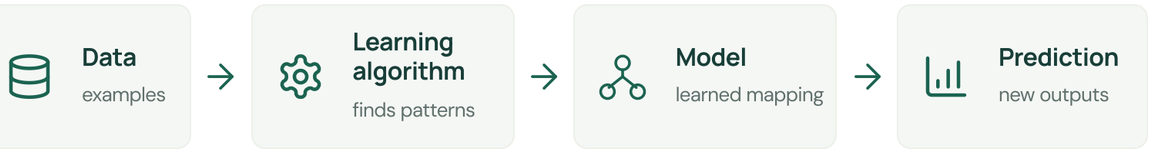
ML fundamentals

What is machine learning? Terminology and learning approaches

What is machine learning?

Machine learning is a branch of AI that enables systems to learn patterns from data and make predictions or decisions without being explicitly programmed for every rule.

Core idea: use data to build a model that generalises to new examples.



Key terminology

Dataset
collection of examples

Feature
input variable

Target
value to predict

Model
learned mapping

Training
fit model on data

Inference
use model on new data

Learning approaches

Supervised
Learn from labelled data

Unsupervised
Find structure without labels

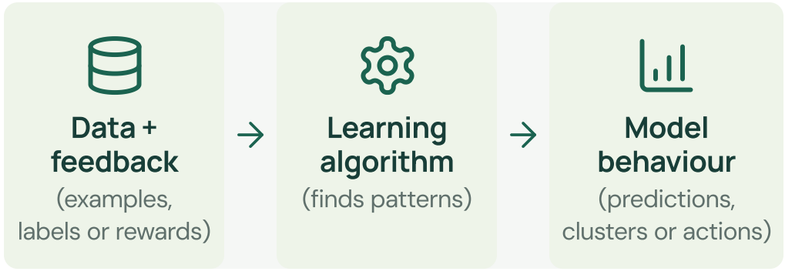
Reinforcement
Learn through rewards and feedback

Main types of machine learning

Supervised, unsupervised, and reinforcement learning

Different learning approaches

Machine learning approaches differ in the kind of feedback they receive. Some learn from labelled examples, some discover patterns without labels, and some learn through rewards and penalties.



Supervised learning

Learns from labelled data to predict a known outcome.

Examples

- Spam detection
- House-price prediction

Unsupervised learning

Finds hidden structure or groups in data without target labels.

Examples

- Customer segmentation
- Clustering similar products

Reinforcement learning

Learns by interacting with an environment and receiving rewards or penalties.

Examples

- Game-playing agents
- Control tasks

Learning type	Training signal	Typical task
Supervised	Known target values	Predict an outcome
Unsupervised	No target labels	Discover patterns or groups
Reinforcement	Rewards from interactions	Learn actions that maximize reward

What is supervised learning?

Learn from labelled examples to predict the target for new data.

Supervised learning is a machine learning approach in which a model learns from labelled examples, meaning each training example contains input features and a known target value. The learned model is then used to predict the correct output for new data.

Labelled training data

House area (m²)	Number of rooms	Age (years)	Price (€)
80	2	5	210,000
120	3	10	280,000
150	4	8	320,000
200	5	15	410,000
...	...	...	...

Training

Find patterns between features and target.

Machine learning model

New data



Area: 140 m²
Rooms: 3
Age: 7 years

Prediction

Estimated price:
€310,000

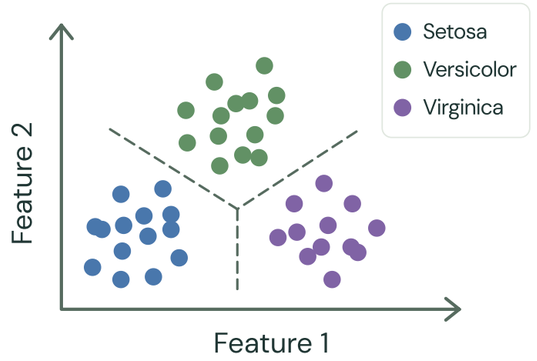
A learning algorithm fits a function f so that $\hat{y} = f(x)$ approximates the correct outcome for a new input. The goal is to make useful predictions on unseen data, not to memorize the training records.

Types of supervised learning

Classification predicts a category, regression predicts a number.

Classification

Predicts a discrete class or label.

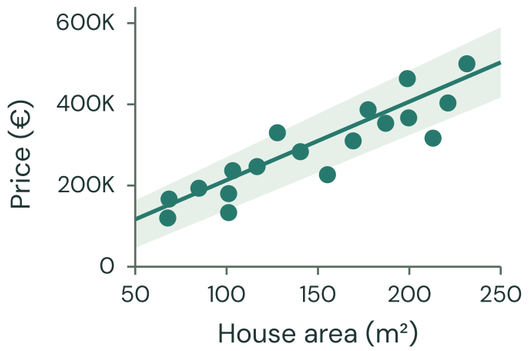


Examples

- Spam detection (Yes / No)
- Customer churn (Yes / No)
- Flower species (Setosa / Versicolor / Virginica)
- Multiple topics for an article (Sports, Finance, Technology)

Regression

Predicts a continuous numeric value.



Examples

- House price in euros
- Electricity demand in megawatts
- Delivery time in minutes
- Product sales forecast

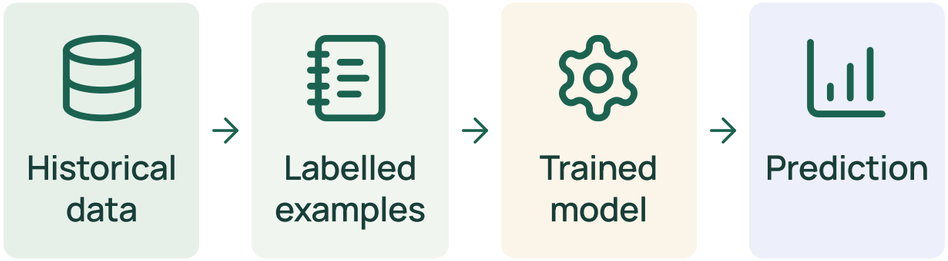
Question	Classification	Regression
What is predicted?	Category or label	Continuous numeric value
Example	Fraudulent or legitimate transaction	Transaction amount
Common models	Logistic regression, decision trees, k-NN classifier	Linear regression, regression trees, k-NN regressor
Example metrics	Precision, recall, F1, ROC-AUC	MAE, RMSE, R^2

Where supervised learning is used

Examples of prediction problems across different domains

From data to decisions

Supervised learning is useful when we have historical examples with known outcomes and want to predict a future or unknown result. It supports decision-making in many fields by learning patterns from labelled data.



Healthcare

Predict patient readmission risk from historical records.

Manufacturing

Predict whether a product will fail quality inspection.

Retail

Forecast demand or identify likely customer churn.

Cybersecurity

Classify suspicious network activity.

Education

Predict students who may need additional academic support.

For each example: ask: What is the target? Which features would be available at prediction time?

What is predictive analytics?

Using historical data to estimate future or unknown outcomes

Definition

Predictive analytics combines historical data, statistical methods, and machine learning to estimate future or otherwise unknown outcomes.

Supervised learning is one of its main techniques, but not every predictive analytics problem requires a complex ML model.



Analytics questions

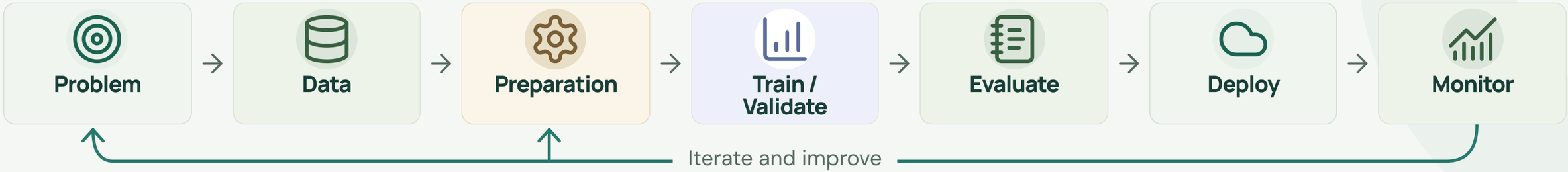
Different types of analytics help us ask and answer different questions.

Descriptive	What happened?	How many customers left last month?
Diagnostic	Why might it have happened?	Which factors are associated with churn?
Predictive	What is likely to happen?	Which customers may leave next month?
Prescriptive	What action should we take?	Which retention intervention should we test?

A predictive model produces an estimate, not certainty. Prediction quality depends on data quality, evaluation methods, and whether future cases resemble past cases.

Predictive analytics process

An iterative workflow from problem definition to monitoring and improvement



1 Define the problem
Decision, target, horizon, success criteria.

2 Collect and understand data
Sources, distributions, missing values, representativeness.

3 Prepare the data
Clean errors, handle missing values, encode categories, create/select features.

4 Split the data
Train/test split and validation strategy; use chronological splits for time-dependent tasks.

5 Establish a baseline
Compare against a simple rule or reference model.

6 Train candidate models
Fit models using training data only, including preprocessing.

7 Evaluate and tune
Use suitable metrics and validation; keep the test set for final unbiased evaluation.

8 Deploy or communicate results
Integrate predictions into decisions or deliver a reproducible report.

9 Monitor and improve
Watch for drift, lower accuracy, fairness concerns, and retraining needs.

The process is iterative: monitoring often leads back to problem framing, new data, and better preparation.

Introducing k-nearest neighbours (k-NN)

Predicting outcomes from the most similar examples

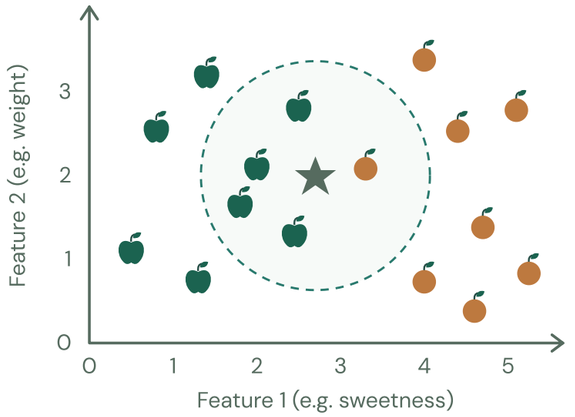
What is k-NN?

k-nearest neighbours (k-NN) is a simple supervised learning algorithm that predicts a class using the k most similar labelled observations in the training dataset.

It is an instance-based, or lazy learning, method because it stores training examples and performs most of its work when a new prediction is requested.

Store training data → **Measure similarity** → **Find k neighbours** → **Vote for a class**

Intuition



- Apple (class 0)
- Orange (class 1)
- New fruit (to classify)
- 5 nearest neighbours (k = 5)

With uniform voting, 4 of the 5 nearest fruits are apples, so predict **apple**.

Illustration uses scaled features.

k-NN classification

Neighbours vote for the predicted class. With uniform voting, choose the class with the most votes.

Example: classify a fruit as an apple or an orange.

How does the choice of k affect results?

1. **Small k, e.g. 1**
   Sensitive to noise and individual examples. Can overfit.
2. **Larger k**
   Smoother boundaries; may miss local patterns and underfit.
3. **Choose k using validation**
   There is no universally best value of k.

With uniform binary voting, odd k avoids vote ties; equal-distance neighbours may still need tie-breaking.

Steps in the k-NN algorithm

From training data to prediction using nearest neighbours

Labelled data
Features (X) and class labels (y)

→

Split + scale
Split first; fit preprocessing on training data only

→

Choose k + metric
Use validation to choose k, metric and voting rule

→

Find neighbours
Compute distances; select the k closest examples

→

Vote
Uniform or distance-weighted class voting

→

Prediction
Output the predicted class for the new observation

Training / preparation

1. Obtain labelled data with features (X) and class labels (y).
2. Split into training, validation and test sets.
3. Fit preprocessing on training data only; scale numerical features.
4. Use validation to choose k, a distance metric and a voting rule.
5. Store the transformed training observations and their labels.

Prediction for a new observation

6. Apply the same fitted preprocessing to the new observation.
7. Calculate distances to training observations, or use an optimised nearest-neighbour search.
8. Identify the k closest neighbours.
9. Combine their labels using uniform or distance-weighted voting.
10. Return the predicted class. Evaluate separately on labelled test data.

Distance metrics

Euclidean distance:

$$d(x, z) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2}$$

Manhattan distance:

$$d(x, z) = \sum_{j=1}^{p} |x_j - z_j|$$

p = number of features; x and z are preprocessed feature vectors.

The metric defines what the model considers similar.

Why scaling matters

Large-scale features can dominate distances. StandardScaler uses training-set means and standard deviations to put non-constant numerical features on comparable scales.



Evaluating supervised learning models

Key metrics for classification and regression, and how to use train, validation, and test data

For classification

The confusion matrix summarises true positives, false positives, true negatives, and false negatives for a defined positive class.

		Predicted class: Positive	Predicted class: Negative
Actual class	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

- **Accuracy** — Fraction of all predictions that are correct.
- **Precision** — Of predicted positive cases, how many are actually positive?
- **Recall** — Of actual positive cases, how many were found?
- **F1-score** — Harmonic mean of precision and recall.

Key formulas

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Accuracy alone can mislead on imbalanced datasets. Predicting “not fraud” for every case may look accurate while detecting no fraud at all.

For regression

- **MAE** — Average absolute prediction error, in target units.
- **RMSE** — Square root of mean squared error; gives more weight to large errors.
- **R^2** — 1 - squared error / mean-baseline squared error. Can be negative.



Training, validation, and test data

Training
Fit model parameters and preprocessing.

→

Validation / cross-validation
Compare models; tune settings such as k.

→

Test
Estimate final performance on unseen examples.

Choose split methods that respect time, groups, and the data-collection process. Random splitting is not appropriate for every predictive task.

k-NN: strengths and limitations

When k-nearest neighbours works well, and where it can struggle

Strengths

- ✓ Easy to understand and explain
- ✓ Works for classification and regression
- ✓ Captures non-linear decision boundaries
- ✓ Relatively few modelling assumptions

Limitations

- ! Sensitive to feature scales and irrelevant features
- ! Prediction can be slow with large training sets
- ! Performance often degrades in high-dimensional spaces
- ! Choice of k, distance metric, and handling of missing values matter

When to try k-NN: As a baseline or practical option for small or medium-sized datasets with meaningful feature distances.

Python demonstration: k-NN classification

Using the Iris dataset with a safe preprocessing pipeline

This example uses scikit-learn's built-in Iris dataset. The pipeline fits preprocessing within each training fold to prevent leakage.

Python code example

```python
# k-NN classification on Iris: all four input features
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Each CV fold fits its own scaler on that fold's training data.
model = make_pipeline(
    StandardScaler(), KNeighborsClassifier(n_neighbors=5)
)
scores = cross_val_score(model, X_train, y_train, cv=5,
                         scoring="accuracy")
print(f"CV accuracy: {scores.mean():.3f} (std {scores.std():.3f})")

# Refit on the training set; evaluate once on the held-out test set.
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"Test accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

```

Conceptual pipeline

Iris data
All four features (X) and labels (y)

→

Train/test split
Stratified split (80/20)

→

StandardScaler
Fit only within each training fold

→

k-NN classifier
k = 5 Euclidean distance

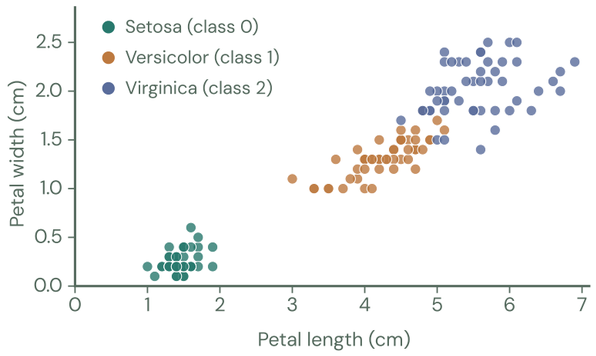
→

Cross-validation / fit
CV on training set; then refit

→

Predict + evaluate
Test accuracy; classification report

Iris dataset: two-feature view



Actual Iris data; the classifier uses all four features.

Discussion

1. What happens if n_neighbors changes from 5 to 1 or 15?
2. Why is scaling placed inside the pipeline?
3. Why is the test set not used to choose k?

Focus on the pipeline and visual intuition here. Use the accompanying Python file for the full walkthrough.

HAMK
Häme University
of Applied Sciences

Thank you

Questions & discussion

Mazhar Mohsin

Häme University of Applied Sciences

Predictive Analytics with Python · Lecture 1 · Supervised Learning